

Steps to Push to code from Working directory to Remote repository:

1. Initiate GIT repository

Command to run:

```
$ git init
```

2. Add a specific file to stag area:

Command to run:

```
$ git add <<file_name>>
```

3. Add all the file available in the working area file to stag area:

Command to run:

```
$ git add .
```

Here . meaning ALL files

4. Commit changes to Local Repository:

Command to run: `git commit -m "All files are committed to git local repo"`

5. Create linkage between Local repo and Remote repo

Command to run:

```
$ git remote add origin https://github.com/etlqalabs/Sep2025Batch.git
```

In this step we are giving a alias name as origin to <https://github.com/etlqalabs/Sep2025Batch.git> which is remote repo.

6. Push changes from Local Repo to Remote Repo

Command to run:

```
$ git push origin master
```

Understanding Git Branches

- **Branch:** A branch in Git represents a separate line of development. The default branch is called main (formerly master), but you can create and switch to other branches for various features or fixes.

- **HEAD:** A pointer that indicates the current branch you're working on and the latest commit on that branch.

2. Creating a New Branch

7. Create a Branch:

`git branch <branch-name>`

- Example: `git branch feature1`
- This creates a new branch named `feature1`, but you're still on the current branch.

8. Create and Switch to a Branch:

`git checkout -b <branch-name>`

- Example: `git checkout -b feature1`
- This creates and switches to the `feature/new-login` branch in one command.

3. Switching Branches

1. Switch to an Existing Branch:

`git checkout <branch-name>`

- Example: `git checkout feature1`

4. Viewing Branches

1. List Local Branches:

`git branch`

2. List Remote Branches:

`git branch -r`

3. List All Branches (Local and Remote):

`git branch -a`

1. Git Merge

What is Git Merge?

`git merge` is used to integrate changes from different branches into a single branch. It combines the content of one branch into another, often the main branch (main or master).

How to Use Git Merge

1. **Check the Current Branch** Before merging, make sure you are on the branch that you want to merge into:

git branch

2. **Switch to the Target Branch** Switch to the branch you want to merge into (e.g., main):

git checkout main

3. **Merge the Branch** Merge the changes from the other branch (e.g., feature-branch) into the current branch:

git merge feature-branch

4. **Resolve Conflicts (If Any)** If there are conflicts, Git will prompt you to resolve them. Open the conflicting files, make the necessary edits, and then:

git add <file>

git commit

Example Workflow

- Create a new branch:

git checkout -b feature-branch

- Make changes, add, and commit:

git add .

git commit -m "Add new feature"

- Switch back to the main branch and merge:

git checkout main

git merge feature-branch

2. Pull Request (It's similar to merge on Remote Repo)

A **pull request** is a way to propose changes, request reviews, and merge code in a collaborative manner. The steps for creating a PR include:

1. Fork/clone the repository.
2. Create a new branch.
3. Make changes and commit.
4. Push the branch to GitHub.
5. Create the pull request.

6. Review and discuss the changes.
7. Merge the PR after approval.

A **Pull Request (PR)** is a way to propose changes to a codebase, request a review, and merge your changes into the main branch of a repository. It's commonly used in Git platforms like GitHub, GitLab, and Bitbucket for collaboration.

Pull Request Overview

- A **PR** is created when a developer wants to merge changes from one branch (usually a feature branch) into another (typically the main or master branch).
- It allows team members to review the proposed changes before they are merged into the main codebase.

Assignments:

Assignment 1:

1. Create a project in pycharm project named “ETLQALabs_Assignments”
2. Create 2 python files for last 2 assignments solution that was shared yesterday with you.
3. Create a Remote repository named “ETLQALabs_Assignments”
4. Push the code to remote repo
5. Create new branch named “OOP” from master branch
6. Add the code above of Student example in a new python file named “OOP_concept”
7. Merge this “OOP” branch with master branch locally and after merging push the master branch code to Remote repository (master)

Assignment 2:

1. Go to “OOP” branch and modify few existing line of code and commit to local repo

2. Switch to “master” branch and modify the few existing line of code and commit changes
3. Try to merge the OOP changes with master branch and see if any merge conflict related issues occurs.
4. If you see any merge conflict then run “ git status” and output will be a file name
5. Edit that file using vim/vim editor and fix the conflicting lines specially remove all the lines not applicable and save the file.
6. Noe try to merge again => It should be able to merge with master branch